

Пояснення архітектури БД

1. USERS

Основна таблиця користувачів.

Навіщо **provider**

Для OAuth:

- local
- google
- apple (future)

Навіщо **role**

RBAC:

- USER
 - PREMIUM
 - ADMIN
-

2. USER_PROFILE

Винесений окремо, щоб не роздувати users.

Зберігає:

- зріст
- вагу
- ціль
- активність
- денну норму КБЖУ

Тут же кешується розрахована норма.

Це швидше, ніж кожен раз перераховувати.

3. FOODS

Найважливіша таблиця.

Усі значення — **на 100 г продукту**.

Приклад:

Куряча грудка:

- calories = 110
- protein = 23
- fat = 1.9

Тоді при 250 г:

110×2.5

23×2.5

created_by

Дозволяє:

- глобальні продукти
- custom foods користувача

4. DIARY_DAYS

Один запис = один день.

Приклад:

2026-05-19

Тут зберігаються total values.

Це optimization layer.

Інакше доведеться постійно aggregate:

Meal → MealItems → Foods

що повільніше.

5. MEALS

Розділення на:

- breakfast
- lunch
- dinner
- snack

Це дозволяє красиво рендерити UI.

6. MEAL_ITEMS

Pivot table.

Зберігає:

grams

calculated macros snapshot

Чому snapshot?

Якщо продукт зміниться в БД, старі записи НЕ повинні ламатись.

Тому при додаванні їжі:

protein

fat

carbs

calories

копіюються в meal_items.

Це production approach.

7. WEIGHT_HISTORY

Для графіків:

- прогрес ваги
 - analytics
 - average trend
-

8. FAVORITE_MEALS

Наприклад:

"Сніданок"

4 яйця

100 г вівсянки

банан

Один клік → додати.

9. SUBSCRIPTIONS

Premium logic.

plan

MONTHLY

YEARLY

status

ACTIVE

CANCELED

EXPIRED

Recommended enums

```
export enum Gender {  
  MALE,  
  FEMALE,  
}
```

```
export enum Goal {  
  CUT,  
  MAINTAIN,  
  BULK,  
}
```

```
export enum ActivityLevel {  
  LOW,  
  MEDIUM,  
  HIGH,  
  VERY_HIGH,  
}
```

```
export enum MealType {  
  BREAKFAST,
```

```
LUNCH,  
DINNER,  
SNACK,  
}
```

```
export enum UserRole {  
  USER,  
  PREMIUM,  
  ADMIN,  
}
```

Що робити далі

1. Згенерувати Prisma schema
2. Продумати relations
3. Написати SQL indexes
4. Зробити auth flow
5. Почати backend modules у NestJS